

L3 Patterns-in-Code Document — HELEP

Part A — Pre-implemented patterns

A.1 Choreographed Saga

Where: The saga spans three services:

- ``sos-service/app/main.py`` — ``trigger()`` publishes ``sos.triggered``
- ``dispatch-service/app/main.py`` — consumes ``sos.triggered``, assigns responder, publishes ``responder.assigned``
- ``notification-service/app/main.py`` — consumes ``responder.assigned``, logs SMS, publishes ``notification.sent``

Compensation step: When citizen calls ``POST /sos/{id}/cancel``, ``sos-service`` publishes ``sos.cancelled``. ``dispatch-service`` consumes it and executes the rollback: sets ``responders.busy=0`` and marks the assignment row as ``RELEASED``. This is in ``dispatch-service/app/main.py`` in the ``sos_cancelled`` handler.

Rollback trigger event: ``sos.cancelled``

Why choreography not orchestration: No central coordinator exists.

Each service reacts to events independently — if ``dispatch-service`` crashes, it replays from Kafka on restart without losing the saga state.

A.2 Pub/Sub via Apache Kafka

Where: ``app/events.py`` in every service — ``AIOKafkaProducer`` (publish) and ``AIOKafkaConsumer`` (subscribe).

Consumer group semantics: Each service has its own ``group_id`` (e.g. ``dispatch-service``, ``notification-service``). This means:

- All 5 services receive the same ``sos.triggered`` event independently
- If dispatch-service has 3 replicas, Kafka splits partitions between them so only ONE replica processes each event — no double dispatch

At-least-once delivery: In ``events.py`` ``consume()``:

``enable_auto_commit=False`` — the consumer only calls

``await consumer.commit()`` AFTER the handler succeeds. If the handler crashes mid-processing, the offset is not committed, so the event is redelivered on restart. This guarantees no SOS is silently dropped.

Partition keying: ``publish(..., key=incident_id)`` in every saga-critical call. Same ``incident_id`` always routes to the same partition → ordering preserved → dispatch-service processes ``sos.triggered`` before ``sos.cancelled`` for the same incident.

A.3 Repository

Where: ``app/db.py`` in every service. All SQL is isolated here.

``main.py`` only calls functions like ``insert_user()``, ``find_by_phone()``, ``add_contact()`` — never writes SQL directly.

Why: If route handlers queried SQLite directly, a schema change would require hunting through all endpoint code. The Repository pattern centralises all data access — change the schema in one place only. It also makes testing easier: mock ``db.py`` and test handlers without a real database.

A.4 Strategy

Where: ``dispatch-service/app/matching.py``

Three interchangeable algorithms all implementing the ``Matcher`` protocol:

- ``NearestMatcher`` — picks responder with smallest haversine distance
 - ``CredibilityWeightedMatcher`` — scores by ``credibility / (distance_km + 1)``
 - ``RoundRobinMatcher`` — assigns in turn regardless of distance
- (added by student)

How to switch: Set environment variable ``MATCHER=nearest``, ``MATCHER=credibility``, or ``MATCHER=roundrobin``. The ``matcher()`` factory function reads this and returns the correct instance.

No if-chains in business logic — open/closed principle satisfied.

Added third strategy: ``RoundRobinMatcher`` added in ``dispatch-service/app/matching.py``. Uses a class-level ``_index`` counter to cycle through available responders fairly. Useful when all responders are equidistant (e.g. indoor deployments).

A.5 Outbox-lite

Where: ``sos-service/app/main.py`` ``trigger()`` function.

The pattern: DB write happens first, then ``await publish(...)`` in the same async block:

```
```python
insert_incident(...) 1. write to SQLite first
await publish("sos.triggered", {...}, key=iid) 2. then publish
```
```

Why "lite": A real Outbox pattern uses a dedicated outbox table in the DB and a separate polling process that reads from it and publishes. This guarantees delivery even if the process crashes between write and publish. Our "lite" version skips the polling process — if the service crashes after the DB write but before ``await publish()``, the event is lost. Acceptable for a dev build;

production would use a full transactional outbox.

A.6 Circuit Breaker

Where: ``app/events.py`` ``class CircuitBreaker`` in every service.

Completed state machine:

- ``CLOSED`` (normal): ``opened_at`` is `None`` → ``allow()`` returns ``True``
- ``OPEN`` (failing): ``fails`` $\geq$ `fail_threshold`` → ``opened_at`` set to current time → ``allow()`` returns ``False``, all publishes rejected
- ``HALF-OPEN`` (testing): ``elapsed`` $\geq$ `reset_after_s`` → ``allow()`` returns ``True`` for one request. If it succeeds → back to `CLOSED`` (``opened_at`` = `None``). If it fails → stays `OPEN``.

State transitions:

- ``record_failure()`` increments ``fails``; trips to `OPEN`` when ``fails`` $\geq$ 5
- ``record_success()`` resets ``fails`=0`` and ``opened_at`=None`` → `CLOSED``
- Time-based reset: after 10 seconds in `OPEN``, one request is allowed through (`HALF-OPEN`` test)

Why it matters: If Kafka goes down, without a Circuit Breaker every ``publish()`` call blocks and times out, cascading the failure to all HTTP handlers. The Circuit Breaker fails fast after 5 failures, returning an immediate error instead of hanging.

Part B — Patterns added

B.1 Bulkhead

Pattern source: Cloud-Native catalogue (Netflix)

Where added: Architecture-level — each service has its own Kafka consumer group (`group\_id` set per service in `events.py` `consume()` call).

Problem solved in HELEP: If `analytics-service` is slow processing events (e.g. heavy aggregation query), without Bulkhead it would slow down the shared consumer, delaying dispatch assignments. By isolating each service into its own consumer group, a slow analytics consumer never blocks dispatch-service from reading `sos.triggered`.

Trade-off: Kafka must deliver each event to multiple consumer groups (one per service) — higher broker load vs. better fault isolation. The trade-off is worth it given ASR-1 (Reliability).

B.2 Idempotency Key

Pattern source: Enterprise Integration Patterns (Hohpe & Woolf)

Where added: `dispatch-service/app/db.py` — the assignment table uses `incident\_id` as PRIMARY KEY. The INSERT uses `INSERT OR IGNORE` semantics.

Problem solved in HELEP: Kafka at-least-once delivery means `sos.triggered` can be redelivered if the consumer crashes after processing but before committing the offset. Without idempotency, dispatch-service would assign two responders to the same incident. The PRIMARY KEY constraint makes the second INSERT a no-op — the assignment is created exactly once regardless of redeliveries.

Trade-off: Requires a meaningful unique key per business event

(incident_id). If IDs were not globally unique, this would fail.

Alternative: pessimistic locking — rejected because it reduces throughput.

Part C — Anti-pattern avoided

Anti-pattern: Shared Database

HELEP explicitly avoids the Shared Database anti-pattern. Each service owns its own isolated SQLite file (``DB_PATH`` env var points to a per-service path). No service reads or writes another service's database directly.

Evidence: ``user-service/app/db.py`` manages only ``users`` and ``contacts`` tables. ``dispatch-service/app/db.py`` manages only ``responders`` and ``assignments``. There are no cross-service JOIN queries anywhere in the codebase.

Why it matters: A shared database couples services at the schema level — a migration in one service's table can break another service at runtime. By isolating databases, each service can evolve its schema independently, satisfying the Portability NFR (SRS §3).